

SEMANA 2 · TEORIA

Model + Repository

CRUD Básico

ORM · JPA · Hibernate · Entidades · Relacionamientos · Spring Data Repository

O que é ORM?

Fundamento

Object-Relational Mapping (ORM) é uma técnica que traduz automaticamente objetos Java em linhas de tabelas SQL — e vice-versa.

Fonte: Hibernate ORM Project · hibernate.org

Elemento Java	Equivalente no BD
@Entity Produto	TABLE produtos
@Id Long id	id BIGINT PRIMARY KEY
String nome	nome VARCHAR(150)

💡 Por quê usar ORM? Sem ORM, cada operação exige SQL manual: código verboso, propenso a erros e acoplado ao banco. O ORM elimina esse boilerplate e deixa você focar na lógica de negócio.



Sem SQL manual
para CRUD básico



Portabilidade
entre bancos



SQL injection
evitado



Lazy/Eager
fetching

Mapeie sua primeira entidade

🕒 15 min

Aplique ORM na prática — agora no seu projeto

1

Crie o pacote

Crie o pacote entity no seu projeto Spring Boot (src/main/java/.../entity)

2

Defina uma entidade simples

Escolha a entidade mais simples do seu projeto (sem relacionamentos). Ex: Categoria, Tipo, Status. Crie a classe com @Entity, @Table, @Id, @GeneratedValue e 2-3 atributos.

3

Adicione @Column

Coloque constraints nas colunas: nullable = false, length, precision. Verifique no H2 Console (/h2-console) se a tabela foi criada corretamente.

4

Confira no H2

Acesse <http://localhost:8080/h2-console> e execute: SHOW TABLES; — sua tabela deve aparecer.

JPA vs Hibernate

Conceito

JPA (Jakarta Persistence API)

📋 É a ESPECIFICAÇÃO

Define interfaces e anotações:

@Entity · @Id · @Column
@OneToMany · EntityManager

Faz parte do Jakarta EE
(antes javax.persistence,
agora jakarta.persistence)

Não tem implementação própria

implements



Hibernate ORM

⚙️ É a IMPLEMENTAÇÃO

Implementa a spec JPA e adiciona
recursos extras:

Lazy loading · Cache L2
HQL/JPQL · Dialetos SQL

Usado automaticamente pelo
Spring Boot via starter-data-jpa

Versão: Hibernate 6.x no Spring Boot 3.x

🎯 Regra de ouro: Escreva sempre contra as interfaces JPA (jakarta.persistence.*) — não use APIs exclusivas do Hibernate. Assim seu código fica portátil entre implementações.

Fonte: docs.spring.io/spring-data/jpa · hibernate.org · Jakarta EE Spec 3.1

Configure seu ambiente Spring Boot

🕒 10 min

Verifique que o JPA + H2 estão funcionando antes de criar entidades

1

Abra o application.yml

Confirme que tem: `datasource.url: jdbc:h2:mem:projetodb` e `jpa.hibernate.ddl-auto: create-drop` e `jpa.show-sql: true`

2

Suba a aplicação

Run → 'Started ...Application in X seconds' deve aparecer no console. Se falhar, verifique as dependências no pom.xml (spring-boot-starter-data-jpa + h2).

3

Acesse o H2 Console

`http://localhost:8080/h2-console` → JDBC URL: `jdbc:h2:mem:projetodb` → Connect. Deve abrir sem erro.

4

Identifique suas entidades

Liste no papel: quais serão as 4+ entidades do seu projeto? Qual é a mais simples (sem relacionamentos)? Essa será a primeira a implementar.

Como transformar uma classe Java em tabela SQL com anotações JPA

Produto.java

```
@Entity
@Table(name = "produtos")
@Data @NoArgsConstructor
public class Produto {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(nullable=false, length=150)
    private String nome;

    @DecimalMin("0.01")
    @Column(precision=10, scale=2)
    private BigDecimal preco;

    @CreationTimestamp
    @Column(updatable = false)
    private LocalDateTime criadoEm;
}
```

- **@Entity**
Marca a classe como tabela JPA
- **@Table(name=...)**
Define o nome da tabela no banco
- **@Id**
Chave primária (PK) da entidade
- **@GeneratedValue**
Auto-incremento gerenciado pelo BD
- **@Column**
Constraints: nullable, length, precision
- **@NotBlank / @Size**
Validação antes de persistir
- **@CreationTimestamp**
JPA preenche automaticamente no INSERT
- **@Data (Lombok)**
Gera getters, setters, equals, toString

Implemente suas 4 entidades JPA

🕒 25 min

Aplique todas as anotações vistas — do @Entity ao @CreationTimestamp

1

Crie as entidades simples primeiro

Entidades sem relacionamentos: Categoria, Tipo, Status, etc. Adicione @Entity, @Table, @Id + @GeneratedValue(IDENTITY), e atributos com @Column(nullable=false).

2

Adicione Lombok

Coloque @Data, @NoArgsConstructor, @AllArgsConstructor em cada entidade. Verifique que o Lombok está no pom.xml como dependência.

3

Adicione Bean Validation

Nas Strings: @NotBlank + @Size(max=150). Nos números: @DecimalMin('0.01'). Datas de criação: @CreationTimestamp + @Column(updatable=false).

4

Verifique no H2 Console

Suba a app e acesse /h2-console. Execute: SHOW TABLES; — as 4 tabelas devem aparecer. Depois: SELECT * FROM CATEGORIAS; para confirmar a estrutura.

💡 Dica: suba a app após cada entidade — não espere criar as 4 de uma vez para testar.

Os 3 Tipos de Relacionamento JPA

Seu projeto precisa usar pelo menos 2 destes 3. Veja como cada um funciona na prática:

@ManyToOne

Mais

Muitos Produtos → UMA Categoria

```
@ManyToOne
@JoinColumn(
    name="categoria_id")
private Categoria categoria;
```

→ Cria coluna categoria_id em produtos

Lado DONO da relação — possui FK

@OneToMany

Lado

Uma Categoria → MUITOS Produtos

```
@OneToMany(
    mappedBy="categoria",
    cascade=CascadeType.ALL)
private List<Produto>
    produtos;
```

→ Não cria coluna nova (usa mappedBy)

Lado INVERSO — use mappedBy

@ManyToMany

Tabela

Produto ↔ MUITAS Promoções

```
@ManyToMany
@JoinTable(
    name="produto_promocao")
private Set<Promocao>
    promocoas;
```

→ Cria tabela intermediária automática

Cria tabela com 2 FKs automática

Crie os Relacionamentos entre suas Entidades

🕒 20 min

Conecte suas entidades com @ManyToOne, @OneToMany e @ManyToMany

1

Adicione @ManyToOne na entidade filha

Ex: em Produto, adicione: @ManyToOne @JoinColumn(name='categoria_id') private Categoria categoria; — isso cria a FK no banco.

2

Adicione @OneToMany na entidade pai

Ex: em Categoria, adicione: @OneToMany(mappedBy='categoria', cascade=CascadeType.ALL) private List<Produto> produtos; — não cria coluna nova.

3

Implemente @ManyToMany se precisar

Ex: um Produto pode estar em muitas Promoções. Use @JoinTable(name='produto_promocao') para criar a tabela intermediária automaticamente.

4

Valide as tabelas no H2 Console

Verifique: SELECT * FROM PRODUTOS; — a coluna categoria_id deve aparecer. Para ManyToMany: SHOW TABLES; — a tabela intermediária deve existir.

Spring Data JPA lê o nome do método e gera o SQL automaticamente — zero código extra necessário

ProdutoRepository.java

```
@Repository
public interface ProdutoRepository
    extends CrudRepository<Produto, Long> {

    // Query Methods – Spring gera o SQL pelo nome:
    List<Produto> findByName(String nome);

    List<Produto> findByNameContainingIgnoreCase(
        String nome);

    List<Produto> findByCategoriaId(
        Long categoriaId);

    List<Produto> findByPrecoBetween(
        BigDecimal min, BigDecimal max);

    // JPQL customizada para casos complexos:
    @Query("""
        SELECT p FROM Produto p
        WHERE p.preco <= :limite
        ORDER BY p.preco DESC
        """)
    List<Produto> abaixoDoLimite(
        @Param("limite") BigDecimal limite);

    boolean existsByNameIgnoreCase(String nome);
}
```

Palavras-chave para Query Methods

Palavra-chave	Exemplo
findBy + Atributo	<code>findByName</code>
Containing	<code>findByNameContaining</code>
IgnoreCase	<code>findByNameIgnoreCase</code>
Between	<code>findByPrecoBetween</code>
LessThan / GreaterThan	<code>findByPrecoLessThan</code>
OrderBy	<code>findBy...OrderByNomeAsc</code>
And / Or	<code>findByNameAndPreco</code>
countBy / existsBy	<code>existsByName</code>

- Entenda: o Spring lê `findByNameContainingIgnoreCase` e gera automaticamente o SQL equivalente com `LIKE` e `LOWER()`. Zero implementação!

Crie os Repositories e Query Methods

🕒 15 min

Um Repository por entidade + pelo menos 1 método derivado cada

1

Crie a interface do Repository

Para cada entidade: `public interface ProdutoRepository extends CrudRepository<Produto, Long> {}` — apenas isso já dá `save()`, `findAll()`, `findById()`, `delete()`.

2

Adicione Query Methods derivados

Pelo menos 1 por entidade. Ex: `findByNomeContainingIgnoreCase(String nome)`, `findByCategoriald(Long id)`, `existsByNomeIgnoreCase(String nome)`.

3

Adicione uma @Query JPQL customizada

Ex: `@Query('SELECT p FROM Produto p WHERE p.preco <= :limite ORDER BY p.preco DESC') List<Produto> abaixoDoLimite(@Param('limite') BigDecimal limite);`

4

Teste no Swagger (após criar o Controller)

`GET /api/v1/produtos?nome=mouse` — deve retornar produtos filtrados. Confirme que os Query Methods do Repository disparam as queries certas via `show-sql: true` no console.

Nesta semana: só GETs. POST/PUT/DELETE chegam na Semana 3 junto com a camada Service.

ProdutoController.java (versão Semana 2)

```
@RestController
@RequestMapping("/api/v1/produtos")
@RequiredArgsConstructor
public class ProdutoController {

    private final ProdutoRepository repository;

    @GetMapping
    public ResponseEntity<Iterable<Produto>> listar() {
        return ResponseEntity.ok(repository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Produto> buscar(
        @PathVariable Long id) {
        return repository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }
}
```

Endpoints funcionando agora:

GET**/api/v1/produtos**

200 OK — lista todos

GET**/api/v1/produtos/{id}**

200 OK ou 404 Not Found



Escopo desta semana: Controller fala direto com Repository. Sabemos que é "errado" — na Semana 3 extraímos a lógica para Services.

Na Semana 3 você vai adicionar:

- POST — criação de recursos
- PUT — atualização completa
- DELETE — remoção com 204
- Service layer + @Transactional

Monte o Controller e Teste no Swagger

🕒 20 min

Valide GET /listar e GET /{id} para todas as suas entidades antes de commitar

1

Crie o @RestController para cada entidade

Adicione @RestController, @RequestMapping('/api/v1/entidades'), @RequiredArgsConstructor. Injete o Repository com final ProdutoRepository repository;

2

Implemente GET /listar e GET /{id}

@GetMapping: return ResponseEntity.ok(repository.findAll()); — @GetMapping('/{id}'): return repository.findById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());

3

Suba e acesse /swagger-ui.html

Execute ./mvnw spring-boot:run. Abra <http://localhost:8080/swagger-ui.html>. Seus endpoints devem aparecer agrupados por Controller. Execute o GET direto da UI.

4

Faça o commit com mensagem semântica

git add . && git commit -m 'feat: add Controllers with GET endpoints for all entities' — Isso evidencia a entrega para a rubrica da Semana 2.

- | | | | |
|--|-------|--|-------|
| ☐ Mín. 4 entidades JPA criadas com @Entity | 8 pts | ☐ 1 Repository por entidade (CrudRepository) | 8 pts |
| ☐ Cada entidade tem @Id e @GeneratedValue | | ☐ 1+ Query Method derivado por Repository | |
| ☐ Colunas com @Column e constraints corretas | | ☐ GETs (listar / por ID) no Swagger funcionam | 8 pts |
| ☐ Pelo menos 1 @ManyToOne implementado | 8 pts | ☐ Retorno 404 correto para ID inexistente | |
| ☐ Pelo menos 1 @OneToMany ou @ManyToMany | | ☐ Commits claros: feat: add Produto entity | |
| ☐ Tabelas aparecem no H2 Console corretamente | | ☐ Repositório GitHub atualizado | |

Acumulado desta semana: ~32 pontos da rubrica | Entidades + Relacionamentos + Repositories + Controllers